Network Intrusion Detection Report

NSL-KDD multi-class intrusion classification using Random Forest

Course	Advanced Python (ICS0019)
Team members	Stefan Stoiana-Mois, Ian Tuttle
Date	23.05.2026
Repository	https://github.com/caramida91/llmtrain.git

1. Approach

The assignment goal was to train a supervised machine learning model for network intrusion detection. Each connection had to be classified into one of five categories: Normal, DoS, Probe, R2L, and U2R. Our first aim was to build a reliable Random Forest baseline, then improve macro F1-score by testing class weighting, number of trees, and maximum tree depth.

We focused on macro F1-score rather than only accuracy because the dataset is imbalanced. Rare classes such as R2L and U2R are important, but they have far fewer records than Normal and DoS traffic.

1.1 Preprocessing

Step	What was done	Reason
Categorical encoding	protocol_type, service, and flag were encoded using LabelEncoder.	Models require numerical input.
Dropped level	The difficulty level column was removed.	It is not a real traffic feature for prediction.
Dropped num_outbound_cmds	This column was removed because it is constant zero.	A constant feature does not help classification.
Attack mapping	Specific attack names were mapped into Normal, DoS, Probe, R2L, and U2R.	The project requires five target classes.
Scaling	No StandardScaler or MinMaxScaler was used.	Random Forest is tree-based and does not require scaling.
Feature engineering	No new features were created.	We first tested whether parameter tuning alone could beat the baseline.

1.2 Class Imbalance Handling

We tested Random Forest both with and without class weighting. The training distribution itself was not changed because no oversampling or undersampling was used. Instead, class_weight='balanced' changed the importance given to minority classes during training. In our tests, class weighting surprisingly did not improve the final macro F1-score, so the final model used class_weight=None.

2. Experiments

We ran five main experiments. The best practical model was Experiment 4. Experiment 5 had a score higher by only 0.0002, but used 1000 trees and took longer, so we selected Experiment 4 as the final model.

#	Description	Algorithm	Imbalance handling	Macro F1 (CV)	Macro F1 (test)
1	Balanced Random Forest	Random Forest	class_weight='balanced'	0.9064 +/- 0.0392	0.4748
2	Basic Random Forest	Random Forest	None	0.9148 +/- 0.0400	0.4938
3	More trees	Random Forest	None	0.9138 +/- 0.0411	0.4956
4	More trees + limited depth	Random Forest	None	0.9169 +/- 0.0374	0.5005
5	1000 trees + limited depth	Random Forest	None	0.9147 +/- 0.0336	0.5007

2. Experiments - Important Observations

Experiment 1: Balanced Random Forest

This was close to the provided baseline. It performed well on common classes, but almost completely failed on R2L and U2R. The final test macro F1-score was 0.4748.

Experiment 2: Basic Random Forest

Removing class_weight='balanced' improved the test macro F1-score to 0.4938. This showed that class weighting was not automatically useful for this dataset and model combination.

Experiment 3: More Trees

Increasing the number of trees gave a small improvement to 0.4956. The model became slightly more stable, but rare attack classes were still difficult.

Experiment 4: More Trees + Limited Depth

This was our selected final model. Limiting the tree depth to 20 while using 500 trees improved generalization and reached a test macro F1-score of 0.5005.

Experiment 5: 1000 Trees + Limited Depth

This achieved 0.5007, but the improvement over Experiment 4 was only 0.0002. Because of the higher training time and almost identical result, Experiment 4 was chosen as the final model.

3. Final Results

3.1 Best Model

Item	Final choice
Algorithm	Random Forest
n_estimators	500
max_depth	20
min_samples_split	2
class_weight	None
random_state	42
n_jobs	-1
Feature engineering	No additional features

3.2 Final Macro F1-Score

Metric	Score
Macro F1 (test)	0.5005
Macro F1 (CV)	0.9169 +/- 0.0374
Accuracy	0.75
Weighted average F1	0.70

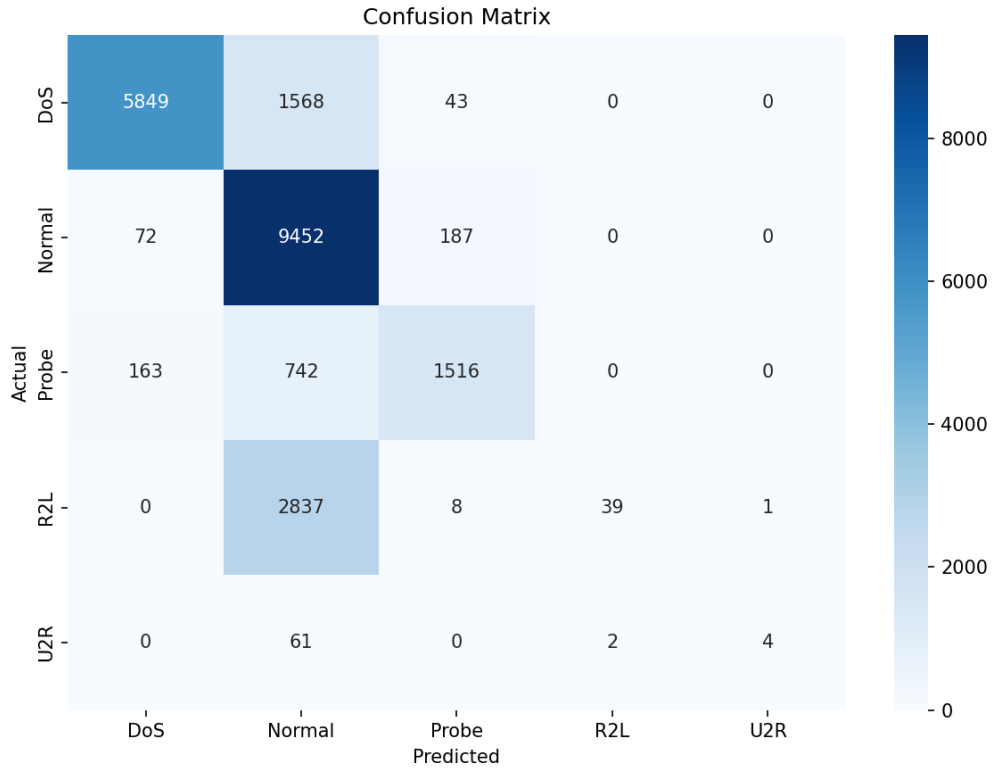
The final model improved over the provided baseline macro F1-score of approximately 0.47. The main improvement came from tuning Random Forest parameters rather than using class weighting.

3.3 Classification Report

Category	Precision	Recall	F1-score	Support
DoS	0.96	0.78	0.86	7460
Normal	0.65	0.97	0.78	9711
Probe	0.86	0.63	0.73	2421
R2L	0.97	0.01	0.02	2885
U2R	0.80	0.06	0.11	67

The model performed well on DoS, Normal, and Probe traffic. However, R2L and U2R remained difficult. The low recall values mean that many of these rare attacks were missed, especially because most R2L samples were predicted as Normal.

3.4 Confusion Matrix



The diagonal cells show correct predictions. DoS, Normal, and Probe have many correct predictions. The R2L row shows the main weakness: most R2L attacks were classified as Normal.

4. Cross-Validation vs. Test Score

Metric	Value
CV macro F1	0.9169 +/- 0.0374
Test macro F1	0.5005
Gap (CV - test)	0.4164

The gap between cross-validation and test performance is large. This is expected to some degree because KDDTest+ contains attack types that are not present in KDDTrain+, so the final test set is harder. It also suggests that the Random Forest learned the training distribution well but did not generalize equally well to unseen attacks.

The largest weakness is recall for R2L and U2R. These categories are rare and can look similar to normal traffic, so the model often predicts Normal instead of the correct rare attack class. Because macro F1 gives equal weight to all classes, these weak rare-class results strongly reduce the final score.

5. What Worked and What Did Not

What worked

- Limiting tree depth while using more trees had the biggest positive impact.
- The final Random Forest with 500 trees and max_depth=20 beat the baseline macro F1-score.
- Removing class_weight='balanced' improved the test score compared with the balanced version.

What did not work

- class_weight='balanced' did not improve R2L or U2R enough and lowered the final score.
- Increasing to 1000 trees gave only a tiny improvement of 0.0002, so it was not worth the extra time.
- The model still missed most R2L and U2R attacks.

Brief Reflection

With more time, we would test XGBoost or LightGBM, apply SMOTE or SMOTEENN for rare classes, tune decision thresholds to increase rare-class recall, and create features such as src_bytes/dst_bytes ratios. Another possible improvement would be a separate binary classifier focused on detecting R2L and U2R before applying the final five-class classifier.

Appendix: Environment

Hardware	Intel Core i9-13900H, RTX 5060 8GB Laptop VRAM, 16GB RAM
Python version	Python 3.13
Key libraries	pandas, numpy, scikit-learn, matplotlib, seaborn, imbalanced-learn, xgboost
Random seed	42